

AGENCY SOCIAL OS

Final Product Requirements + System Architecture + Implementation Specification

Version 1.0 — Consolidated Final Blueprint

Document	Final implementation baseline
Product stage	Internal agency MVP → multi-client portal → future SaaS
Architecture	Modular monolith + background workers
Core stack	Next.js + NestJS + PostgreSQL + Redis/BullMQ
Budget constraint	■1,000–■1,500 maximum initial spend
First provider	LinkedIn PoC; verify current official API requirements before implementation

This document consolidates the original PRD/architecture, Architecture V2, and Implementation Specification V1, then applies the current hard budget constraint. It is intended to be the engineering contract for implementation.

1. Document Control & Source Reconciliation

The original product/architecture document established the problem, vision, personas, functional requirements, platform scope, initial stack, roadmap, and risks. Architecture V2 simplified the product hierarchy, introduced a provider adapter boundary, hybrid analytics, stronger OAuth/token security, isolated queues, and a modular-monolith direction. Implementation Specification V1 translated those decisions into repository, database, OAuth, queue, API, testing, and deployment details.

The final specification preserves those strong decisions while correcting several implementation-level issues: capability data is treated as observed/cacheable state rather than permanent truth; scaling is triggered by measured bottlenecks rather than a fixed account count; OAuth security is provider-aware; UUIDv7 generation must be explicitly chosen; and the old paid-infrastructure estimate is replaced by a free/local-first MVP strategy.

Source	What is retained
Original architecture/PRD	Vision, personas, requirements, platform scope, roadmap and risks.
Architecture V2	Organization → Workspace → Account, provider adapters, hybrid analytics, encrypted tokens, isolated que
Implementation Spec V1	NestJS modules, Prisma baseline, tenant guard/extension, OAuth state, AES-GCM, BullMQ, API contract, t
Current founder constraint	■1,000–■1,500 hard MVP budget; no paid infrastructure dependency.

API verification rule: provider capabilities are time-sensitive and some source-document rows are inconsistent. Before implementing any provider, verify current official documentation, account types, scopes, approval requirements, quotas, token behavior and supported operations.

2. Product Vision & Strategy

Agency Social OS is a unified operating system for a small digital agency to manage multiple social-media accounts from one place. The product combines account connections, analytics, content operations, publishing, scheduling, reporting, team permissions, and future client access.

The strategic path is intentionally staged: first prove the product internally, then expose selected read-only functionality to clients, and only after operational proof consider commercial SaaS. This prevents premature billing, white-labeling, distributed infrastructure, and expensive API commitments.

The primary value proposition is a single pane of glass for social operations while eliminating password sharing and manual reporting. The platform must be more secure and cheaper to operate than a collection of spreadsheets, screenshots, and separate platform logins.

3. Personas & User Outcomes

Persona	Needs	Initial role
Agency Owner	Workspace overview, account health, analytics, team control	OWNER
Social Media Manager	Deep analytics, content, scheduling, publishing	MANAGER
Editor	Create/edit content and publish where permitted	EDITOR
Client Approver	Approve drafts and view performance	CLIENT_APPROVER
Client/Viewer	Read-only analytics and scheduled content	VIEWER
Admin	System/user/integration oversight	OWNER initially

Core journey — connect an account

Workspace → Connect Platform → OAuth authorization
→ callback/state validation → code exchange
→ account discovery → account selection
→ encrypted token storage → initial sync job
→ account appears in dashboard

Core journey — analytics

Scheduler → queue:sync → Provider Adapter
→ token retrieval → Social API → raw response
→ normalize → validate → deduplicate/upsert
→ PostgreSQL → cache invalidation → dashboard

4. Functional Requirements

ID	Requirement	Priority
FR-01	Secure email/password authentication and sessions	P0
FR-02	Workspace creation and membership	P0
FR-03	Granular RBAC permissions	P0
FR-04	OAuth connect/disconnect for supported platforms	P0
FR-05	Encrypted OAuth credentials; never exposed to frontend	P0
FR-06	Initial + scheduled analytics sync	P0
FR-07	Unified workspace/account analytics dashboard	P0
FR-08	Capability-aware UI and backend enforcement	P0
FR-09	Provider-aware retries/rate limiting	P0
FR-10	Audit critical actions	P1
FR-11	Publishing and scheduling	P1
FR-12	Post-level analytics	P1
FR-13	Client read-only portal	P1
FR-14	PDF/CSV reporting	P2
FR-15	Comments/DM aggregation	P2
FR-16	Billing/Stripe/white-label SaaS	P2

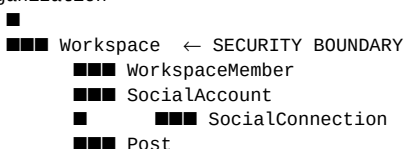
5. Non-Functional Requirements

5. Non-Functional Requirements

Area	Final requirement
Security	No plaintext OAuth tokens; workspace isolation; CSRF/IDOR protection; secret-free logs.
Reliability	Provider failures are isolated in workers and never crash dashboard requests.
Performance	Dashboard reads stored/cached analytics; no live social API calls during page rendering.
Freshness	Daily historical/account sync; more frequent active-post sync only where provider APIs support it.
Scalability	Scale workers when queue/API/database metrics show a bottleneck.
Maintainability	Provider logic isolated in framework-agnostic adapters.
Cost	MVP must stay within █1,000–█1,500 initial spend.

6. Final Domain & Tenancy Model

Organization

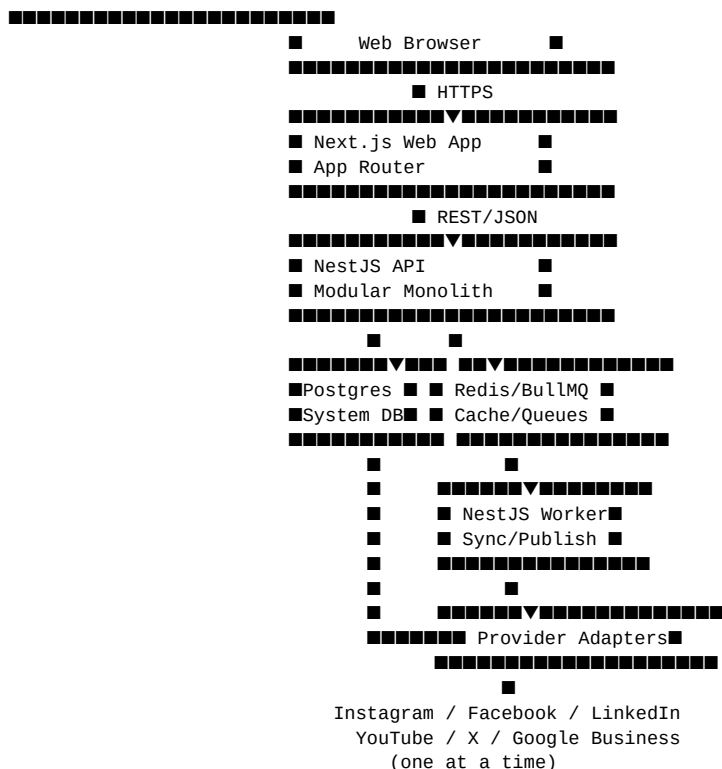


- PostPlatformVariant
- AccountMetricDaily
- AuditLog
- Sync/Webhook operational records

The original hierarchy Agency → Client → Brand → Account was intentionally flattened in Architecture V2. The final model is Organization → Workspace → Social Account. Organization remains the future billing/top-level administrative boundary. Workspace is the practical operational and security boundary for the MVP.

A future client/brand concept can be introduced without breaking the core architecture if actual product usage proves it is needed.

7. Final System Architecture



The web/API/worker processes remain one codebase with strict module boundaries. Microservices are deliberately rejected for the MVP.

8. Technology Stack

Layer	Final choice	MVP rule
Frontend	Next.js App Router + Tailwind + shadcn/ui	Free hosting where practical
Backend	NestJS + TypeScript	Modular monolith
Database	PostgreSQL + Prisma	Free tier/local Docker
Cache/Queue	Redis + BullMQ	Free tier/local Docker
Validation	Zod/shared contracts	No unnecessary duplication
Provider SDK layer	Pure TypeScript adapters	No NestJS/Next.js dependency
Object storage	Deferred until media publishing requires it	0 initially
Monitoring	Structured logs; free error tracking if affordable	No paid dependency
Monorepo	Turborepo	Single repository

9. Monorepo & Module Boundaries

```
agency-os/  
■ ■ ■ apps/  
■ ■ ■ ■ web/  
■ ■ ■ ■ api/  
■ ■ ■ ■ worker/  
■ ■ ■ packages/  
■ ■ ■ ■ database/  
■ ■ ■ ■ providers/  
■ ■ ■ ■ shared/  
■ ■ ■ ■ config/  
■ ■ ■ ■ types/  
■ ■ ■ docker-compose.yml  
■ ■ ■ package.json  
■ ■ ■ turbo.json
```

API modules

```
apps/api/src/  
■ ■ ■ core/  
■ ■ ■ modules/  
■ ■ ■ ■ auth/  
■ ■ ■ ■ workspaces/  
■ ■ ■ ■ members/  
■ ■ ■ ■ integrations/  
■ ■ ■ ■ accounts/  
■ ■ ■ ■ analytics/  
■ ■ ■ ■ publishing/  
■ ■ ■ ■ reports/  
■ ■ ■ ■ webhooks/  
■ ■ ■ ■ audit/
```

Boundary rule: web and API may import shared packages but never import each other directly. The provider package must remain framework-agnostic so provider tests can run without NestJS or Next.js.

10. Authentication, RBAC & Tenant Isolation

Authentication is session-based for the MVP. The browser receives a secure HttpOnly cookie. Passwords are hashed. Google login, MFA and advanced identity features remain future extensions.

Permission	OWNER	MANAGER	EDITOR	CLIENT_APPROVER	VIEWER
workspace.view	✓	✓	✓	✓	✓
members.manage	✓	✓	–	–	–
accounts.view	✓	✓	✓	✓	✓
accounts.connect	✓	✓	–	–	–
accounts.disconnect	✓	✓	–	–	–
analytics.view	✓	✓	✓	✓	✓
posts.create/edit	✓	✓	✓	–	–
posts.publish	✓	✓	✓	–	–
posts.approve	✓	✓	–	✓	–
audit.view	✓	✓	–	–	–

Request
→ AuthGuard
→ WorkspaceGuard (membership)
→ PermissionGuard
→ trusted workspace context
→ service
→ workspace-scoped repository
→ database

Application-level workspace scoping is the primary enforcement mechanism. PostgreSQL RLS can remain a defense-in-depth failsafe where the chosen Prisma/connection strategy supports it safely.

11. Social Provider Adapter Architecture

```
packages/providers/
  ■■■ core/
  ■ ■■■ interfaces/
  ■ ■■■ types/
  ■ ■■■ errors/
  ■■■ linkedin/
  ■■■ facebook/
  ■■■ instagram/
  ■■■ youtube/
  ■■■ x/
  ■■■ google-business/

interface ISocialProvider {
  getAuthorizationUrl(input): string;
  exchangeAuthorizationCode(input): Promise<OAuthCredentials>;
  getProfiles(credentials): Promise<SocialProfile[]>;
  getAccountMetrics(credentials, account): Promise<RawAccountMetrics>;
  getPostMetrics?(credentials, post): Promise<RawPostMetrics>;
  publishPost?(credentials, payload): Promise<PublishResult>;
  deletePost?(credentials, externalPostId): Promise<void>;
  getCapabilities?(context): Promise<ProviderCapabilities>;
  handleWebhook?(request): Promise<NormalizedWebhookEvent>;
}
```

Provider-specific HTTP clients, scopes, payload mapping, rate limits and quirks live inside the adapter. Core analytics/publishing services never contain platform-specific API calls.

12. Capability Resolution

```
Provider definition
+ account type
+ granted scopes
+ approvals/tasks
+ provider response
↓
Capability Resolver
↓
Effective capabilities
ANALYTICS_READ
ACCOUNT_READ
POST_READ
POST_PUBLISH
POST_SCHEDULE
COMMENTS
DMS
WEBHOOKS
```

Stored capabilities are observed/cacheable state, not the permanent source of truth. The backend enforces capabilities on every operation; the frontend only uses them to improve UX.

13. OAuth & Token Security

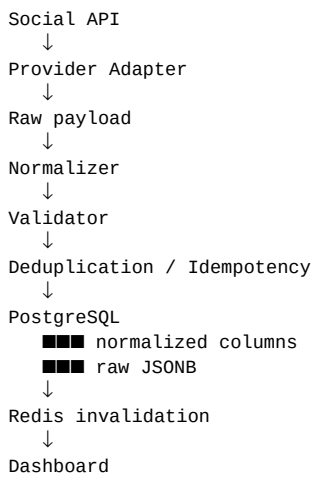
```
Connect
→ short-lived state
→ PKCE verifier/challenge when applicable
→ Redis state record
→ provider authorization
→ callback
→ state validation
→ code exchange
→ account discovery
→ account selection
→ encrypt credentials
→ persist connection
→ queue initial sync
```

Security rules

- State is random, short-lived, single-use and bound to the initiating user/workspace.

- PKCE is used when supported/required.
- Nonce is used when applicable; do not blindly require it for every provider.
- Long-lived tokens never reach the browser.
- AES-256-GCM with a unique IV/nonce per encryption operation.
- Store IV, authentication tag and encryption key version.
- Encryption keys live outside the database.
- Only the integration layer may decrypt tokens immediately before provider calls.
- Tokens, authorization codes and encryption keys are never logged.

14. Analytics Architecture



Data	Final MVP treatment
Account daily metrics	PostgreSQL indexed table; unique account+date.
Raw provider data	JSONB with limited retention/pruning.
Post metrics	Phase 2; provider-dependent.
Aggregates	Calculated from stored data; cache expensive queries.
Time-series DB	Not needed until measured bottleneck.

The fixed 10,000-account migration threshold from the earlier V2 is removed. Scale based on measured ingestion volume, storage growth, query latency and retention requirements.

15. Database Design

Table	Purpose	Important constraints
User	Identity/authentication	Unique email
Organization	Top-level future billing boundary	Simple in MVP
Workspace	Security/operational boundary	Soft delete
WorkspaceMember	Membership + role	Unique workspace+user
SocialAccount	External profile/page/channel	Provider + external ID uniqueness
SocialConnection	Encrypted OAuth credentials	1:1 with SocialAccount
Post	Workspace content	Workspace scoped
PostPlatformVariant	Per-account publishing state	Independent status
AccountMetricDaily	Daily normalized analytics	Unique account+date

Table	Purpose	Important constraints
WebhookEvent	Inbound idempotency	Unique event ID where available
SyncRun	Operational sync diagnostics	Status/counts/errors
AuditLog	Security history	Workspace scoped where applicable

```
User < WorkspaceMember > Workspace
    < SocialAccount < SocialConnection
        < AccountMetricDaily
            < Post < PostPlatformVariant > SocialAccount
                AuditLog / SyncRun / WebhookEvent
```

16. Prisma & ID Strategy

The previous implementation specification described UUIDv7 while using a generic `gen_random_uuid()` default. The final implementation must choose one explicit UUIDv7 generation mechanism compatible with the selected Prisma/PostgreSQL versions and test it before Phase 1 is declared complete.

```
model SocialAccount {
  id          String @id @db.Uuid
  workspaceId String @db.Uuid
  provider    Provider
  externalId  String
  name        String
  avatarUrl   String?
  capabilities String[]
  status      AccountStatus @default(ACTIVE)

  workspace Workspace @relation(fields: [workspaceId], references: [id])
  connection SocialConnection?
  metrics     AccountMetricDaily[]
  variants    PostPlatformVariant[]

  @@unique([provider, externalId])
  @@index([workspaceId, provider])
}

model SocialConnection {
  id                String @id @db.Uuid
  socialAccountId   String @unique @db.Uuid
  encryptedAccessToken String
  encryptedRefreshToken String?
  iv                String
  authTag           String
  expiresAt         DateTime?
  keyVersion        Int @default(1)

  account SocialAccount @relation(fields: [socialAccountId], references: [id], onDelete: Cascade)
}
```

17. Queues, Sync & Reliability

Queue	Purpose	Initial strategy
queue:sync	Initial sync, daily analytics, backfill	Conservative concurrency + provider-aware retry
queue:publish	Scheduled publishing	High priority + idempotent variants
queue:system	Webhooks/notifications/maintenance	Fast acknowledgment

- Queue payloads contain identifiers, never OAuth secrets.
- Jobs must be safe to retry.
- 429 triggers backoff and provider rate coordination.
- 401/token expiry does not retry forever; connection becomes reconnect-required.
- Provider-specific concurrency prevents one provider from blocking others.
- Historical backfills use low concurrency and can be paused.

- Idempotency keys use account/date, provider event ID, or another stable operation key.

Daily sync:
Scheduler → queue:sync → account job
→ provider API → normalize → upsert
→ invalidate workspace analytics cache

18. Publishing & Partial Success

Create Post → Draft → create platform variants
→ validate → schedule → queue:publish
→ provider adapter → publish
→ store externalPostId → update variant status

If A succeeds and B fails:
A = PUBLISHED
B = FAILED
Parent = PARTIAL/PUBLISHING

Publishing is Phase 2 under the budget constraint. Do not purchase media storage or paid APIs before the analytics MVP is proven.

19. REST API Contract

Method	Path	Permission	Purpose
POST	/api/v1/auth/login	Public	Login
POST	/api/v1/auth/logout	Authenticated	Logout
GET	/api/v1/workspaces	Authenticated	List workspaces
GET	/api/v1/workspaces/:id/accounts	accounts.view	List accounts
POST	/api/v1/workspaces/:id/oauth/:provider/connect	accounts.connect	Start OAuth
GET	/api/v1/oauth/:provider/callback	OAuth state	Complete OAuth
GET	/api/v1/workspaces/:id/analytics/overview	analytics.view	Overview
GET	/api/v1/workspaces/:id/analytics/accounts/:id	analytics.view	Account detail
POST	/api/v1/workspaces/:id/posts	posts.create	Create draft
POST	/api/v1/workspaces/:id/posts/:id/schedule	posts.publish	Schedule
POST	/api/v1/webhooks/:provider	Provider signature	Webhook

```
{
  "error": {
    "code": "PROVIDER_TOKEN_EXPIRED",
    "message": "The social connection needs to be reauthorized.",
    "requestId": "req_123"
  }
}
```

Never expose raw provider stack traces, access tokens, refresh tokens, authorization codes, or internal database details.

20. Frontend Architecture

```
apps/web/src/
  app/
    (auth)/login/
    (dashboard)/[workspaceId]/
      page.tsx
      accounts/
      analytics/
      posts/
      calendar/
      settings/
    components/
    lib/
    api-client.ts
    query-client.ts
```

■■■ store/
■■■ ui-store.ts

TanStack Query handles server state. Zustand is limited to local UI state. Permission-aware rendering improves UX, but API authorization remains authoritative.

21. Security Threat Model

Threat	Control
IDOR / tenant leak	WorkspaceGuard + scoped repositories + DB defense-in-depth + tests
OAuth CSRF	Short-lived state + provider-appropriate PKCE
Token theft	AES-256-GCM + external key + no frontend/log exposure
Webhook spoofing	Provider signature validation
Replay	Event IDs/idempotency records
Privilege escalation	Permission checks on mutations
SQL injection	Prisma parameterization
Rate abuse	API rate limit + provider token bucket
SSRF	Strict allowlisting; avoid arbitrary server-side URL fetching
Secret leakage	Redacted logs + isolated secrets

22. Caching, Rate Limiting & Observability

Caching

- Cache expensive analytics aggregations only.
- Every analytics cache key contains workspace ID.
- Invalidate after successful sync.
- Never cache OAuth tokens or sensitive PII.

Rate limiting

- API endpoints use application-level rate limits.
- Provider requests use Redis token-bucket coordination.
- 429 causes backoff; 401 triggers token handling/reconnect.

Observability

- Structured Pino logs.
- requestId, workspaceId, socialAccountId and jobId where safe.
- Queue depth and failed jobs are operational metrics.
- Use free-tier error tracking only if it fits the budget.
- Never log credentials or secrets.

23. Testing & Acceptance Criteria

Layer	Required tests
Unit	Normalizer, capability resolver, encryption, provider mappings
Integration	API routes, repositories, queue processors

Layer	Required tests
Security	Cross-workspace access, permission bypass, token exposure
OAuth	State mismatch, expired state, callback failure, token encryption
Provider	Mocked 200/401/429/5xx responses
E2E	Login → workspace → OAuth → sync → dashboard
Publishing	Idempotency, partial success, retry and scheduling

Critical acceptance tests

- Workspace A cannot access Workspace B data.
- Viewer cannot connect/disconnect accounts.
- Editor cannot manage members.
- OAuth state mismatch aborts callback.
- Tokens are encrypted at rest and never returned.
- 429 triggers backoff.
- 401 transitions the connection to reauthorization-required.
- Repeating the same account/date sync does not duplicate metrics.
- One successful platform variant remains successful if another variant fails.

24. ■1,000–■1,500 MVP Budget & Deployment

The older architecture estimated about \$105/month for a hosted MVP. That assumption is incompatible with the current budget. The final MVP is therefore free-tier/local-first and vendor-portable.

Component	MVP strategy	Initial target
Domain	Optional; use free deployment URL first	■0
Frontend	Free hosting tier	■0
API	Free/low-cost tier or local during PoC	■0 initially
Worker	Free/low-cost tier or combined process where viable	■0 initially
PostgreSQL	Free managed tier or Docker local	■0 initially
Redis	Free managed tier or Docker local	■0 initially
Object storage	Deferred until publishing requires it	■0 initially
Monitoring	Logs + free tier if affordable	■0
X/Twitter	Exclude paid API dependency	■0
Domain later	Only after first demo/client	Within ceiling if desired

Hard rule: the MVP must not require paid Redis, paid database, paid monitoring, AWS S3, paid X/Twitter tiers, billing infrastructure, or a paid email platform. If a feature requires one of these, move it to a later phase.

25. Development Phases

Phase	Goal	Exit condition
0	Repo + Docker + tooling	All apps/packages build locally
1	DB + tenancy	Workspace CRUD + isolation tests
2	Auth + RBAC	Secure login and permission checks
3	Provider core	ISocialProvider + capabilities

Phase	Goal	Exit condition
4	First OAuth provider	Verified OAuth + account connection
5	Analytics	Initial/daily sync + dashboard
6	Hardening	Security + rate limits + audit + staging
7	Publishing	Drafts + variants + scheduling
8	More providers	One provider at a time
9	Client portal	Read-only client experience
10	Commercial SaaS	Billing + white-label + scale

26. Phase 0 File Blueprint

```

agency-os/
├── apps/
│   ├── web/
│   └── api/
│       ├── src/
│       │   ├── core/
│       │   ├── modules/
│       │   ├── auth/
│       │   ├── workspaces/
│       │   └── members/
│       └── worker/
├── packages/
│   ├── database/
│   ├── providers/
│   ├── shared/
│   ├── config/
│   └── types/
├── docker-compose.yml
├── .env.example
├── package.json
└── turbo.json

```

Phase 0 acceptance: local Postgres and Redis run; Prisma migration/seed succeeds; web/API/worker start; lint/typecheck pass; no secrets committed.

27. Architecture Decision Records

ADR	Decision	Reason
001	NestJS modular monolith	Small team, strong module boundaries, no distributed overhead
002	PostgreSQL	Relational tenancy, constraints, indexes, JSONB
003	Redis + BullMQ	Simple background processing + rate-limit coordination
004	REST	Simple caching, webhooks and implementation
005	Workspace security boundary	Simple operational context
006	Application tenancy + DB defense-in-depth	Practical with Prisma + stronger isolation
007	AES-256-GCM	Secure OAuth credential storage
008	Provider adapters	Contain third-party API volatility
009	Normalized + raw JSONB	Unified analytics without data loss
010	Metrics-driven scaling	Avoid arbitrary account thresholds
011	Free/local-first MVP	Hard █1,000—█1,500 budget

28. Risks & Mitigations

Risk	Severity	Mitigation
Provider approval denied	High	Start with one provider; minimum scopes; verify before coding
Token expiry/revocation	High	Refresh where supported; detect 401; reconnect flow
Rate limits	High	Per-provider queues, token bucket, backoff
Analytics differences	High	Adapters + normalized core + raw JSONB
Budget exceeded	Critical	Free/local-first; no paid X; defer S3/billing
Cross-tenant leak	Critical	WorkspaceGuard + repository scoping + security tests
Over-engineering	High	No microservices/Kafka/ClickHouse/Kubernetes in MVP

29. Open Decisions Before Coding

These are implementation choices, not reasons to redesign the architecture.

- Revalidate current official OAuth scopes, account types and review requirements for the first provider.
- Select the exact maintained session/auth implementation while keeping the API contract stable.
- Select and test the UUIDv7 generation mechanism.
- Choose actual free-tier deployment vendors available when implementation begins.
- Decide whether a domain is worth spending part of the ■1,000—■1,500 budget on before the first demo.
- Confirm whether the first provider needs personal profiles, organization pages, or both.

30. MVP Definition of Done

Area	Done when
Repository	Monorepo builds; lint/typecheck pass; local services work
Database	Migrations, constraints and indexes tested
Auth	Login/logout/session works securely
Tenancy	Cross-workspace access blocked and tested
RBAC	Protected mutations require permissions
OAuth	First provider connection works using verified current requirements
Token security	Credentials encrypted and never exposed
Analytics	Initial + scheduled sync works idempotently
Dashboard	Reads stored/cached analytics, not live APIs
Queues	Retry/rate-limit behavior is observable
Deployment	Runs within available budget

31. Final Build Blueprint

```

PRD
↓
Organization → Workspace → Social Accounts
↓
Auth → RBAC → Workspace Isolation
↓
Provider Adapter → OAuth → Encrypted Tokens
↓
PostgreSQL → Normalized Metrics + Raw JSONB
↓
Redis + BullMQ → Sync / Publish / System
↓
Next.js Dashboard

```

↓

Tests → Security → Staging

↓

More Providers → Publishing → Client Portal → SaaS

Final recommendation: stop redesigning and start Phase 0. The architecture is strong enough for future SaaS but small enough for an individual founder to implement. The first commercial objective is a working product demonstrated to a real local business; paid infrastructure should follow revenue, not precede it.

END OF FINAL SPECIFICATION